SYMBIOSIS INSTITUTE OF TECHNOLOGY, HYDERABAD
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade

PROJECT REPORT

ON

"CIVICDESK:

Public Grievance Redressal System"

Subject: Data Structures

Submitted By:

Name	Branch	PRN
Shashi Kiran	CSE	25070721045
Chandra Shekhar	CSE	25070721055

INDEX

1. Acknowledgement
2. Abstract and Introduction
3. Methodology
4. Data Structures and Algorithms
5. Implementation Details
6. Complexity Summary
7. Testing and Verification
8. Benchmark Results
9. Sample Outputs
10. Limitations and Future Scope
11. Conclusion

ACKNOWLEDGEMENT

We take this opportunity to express our heartfelt gratitude to everyone who has guided and supported us throughout the completion of our project titled CivicDesk: Public Grievance Redressal System.

First and foremost, we extend our deepest thanks to [add faculty name], our professor and project guide, for his/her constant encouragement, valuable guidance and continuous support. The feedback and in-depth knowledge shared during the course helped us understand the working of the data structures and algorithms used here and bring our idea to life as a fully functional C program.

We would like to express our sincere gratitude to Dr. Rajinikanth Aluvalu, Director of Symbiosis Institute of Technology, Hyderabad, for providing us with the infrastructure, resources and an environment that encourages innovation, experimentation and learning.

Finally, we would like to thank our peers and classmates for their helpful suggestions, cooperation and moral support throughout the project. We are also thankful to Symbiosis Institute of Technology, Hyderabad, for providing us with the platform to turn our ideas into a working project.

ABSTRACT AND INTRODUCTION

CivicDesk is a menu-driven complaint-management application and data-structures project written in standard C99. Citizens' complaints are registered, prioritised through three linked-list queues, tracked through a per-complaint status history, indexed in a binary search tree and reported with classical searching and sorting algorithms. Manual status updates can be reversed with a linked-list-based undo stack.

Every data structure is built by hand with `malloc` and pointers; no external libraries or ready-made containers are used. All names, areas and complaints in the sample data are fictional, and nothing is sent to any real government system.

Problem. In a typical grievance system, urgent complaints can wait behind routine ones, citizens cannot see what happened to a complaint, and finding or ranking complaints becomes slow as the number grows.

Goal. CivicDesk serves complaints by priority, records every step of their life, and finds and ranks them quickly. It applies queues, singly linked lists, stacks, binary search trees, linear and binary search and five classical sorting algorithms to a realistic public-service problem.

Project links.

- Repository: https://github.com/pulegarisashikirran/civicdesk-grievance-system
- Live demo: https://pulegarisashikirran.github.io/civicdesk-grievance-system/

The live demo is a JavaScript version of the same logic, made for easy demonstration in a browser. The C program in the repository is the main project.

METHODOLOGY

1. How a Complaint Flows Through the System

1. A citizen's complaint is entered through **File a new complaint**. A random unused 4-digit ID is generated; if all 9,000 IDs are occupied, registration is rejected instead of retrying forever.
2. The complaint is inserted into the BST and placed in the Urgent, High or Normal queue. Its status list starts with "Filed".
3. **Process next complaint** takes the first complaint from the highest non-empty queue and appends "Assigned" to its status list.
4. **Update complaint status** moves a processed complaint to "In Progress" or "Resolved". The new status is appended to the linked list and a record is pushed on the undo stack. Complaints still waiting in a queue cannot be updated, so the queues always stay consistent.
5. If a wrong status was entered, **Undo** pops the stack and removes that status.
6. **Search** finds complaints by ID (BST or binary search) or by name or area (linear search), and **View status history** prints the full trail.
7. **Sort and view reports** ranks complaints by the chosen field with the chosen algorithm and shows how long it took.

2. How the Structures Work Together

The BST is the master index of all complaints. The three queues hold only the complaints still waiting to be processed. Each complaint carries its own status linked list, and the undo stack remembers recent status updates so they can be reversed. A plain array of all complaints supports linear search and the sorting reports. Binary search runs over the BST's in-order array. `grievance_system.c` owns all of these and frees them together on exit.

3. Pending Days and Status Tracking

CivicDesk tracks how long every complaint has been open. Dates are stored as day numbers (days since 1 January 1970) and converted to `YYYY-MM-DD` only for display.

- **Open complaints:** pending days = today's day number − the day the complaint was filed. The value is recalculated every time it is displayed or used as a sort key.
- **Resolved complaints:** once the latest status is "Resolved", counting stops at the day of that status, so pending days = resolution day − filing day and resolved complaints no longer look overdue.

- **Live visibility:** the main menu always shows how many complaints wait in each queue. Every table shows ID, name, area, department, priority, current status and pending days. The status history shows the dated trail, including remarks entered by the officer.

DATA STRUCTURES AND ALGORITHMS

CivicDesk is built around six foundational data structures and classical algorithms. Each one is used for the job it is best suited to.

Quick Reference Matrix

Topic	File	Structure / Function	Role in CivicDesk	Complexity
Linear Search	`search_sort.c`	`linear_search()`	Finds complaints by citizen name or area in the full array	O(n) time, O(1) space
Binary Search	`search_sort.c`	`binary_search()`	Finds a complaint by ID in the array sorted by ID	O(log n) time, O(1) space
Singly Linked List	`status_list.c`	`StatusNode`, `StatusHistory`	Chronological status trail of each complaint	O(1) append, O(k) remove last, $k \leq 4$
Queue (linked list)	`queues.c`	`QueueNode`, `LinkedQueue`	Strict first-in, first-out waiting line	O(1) enqueue/dequeue, O(n) space
Priority handling (3 queues)	`queues.c`	`PriorityQueueManager`	Urgent before High before Normal, FIFO inside each level	O(1) per operation
Stack (linked list)	`undo_stack.c`	`StackNode`, `UndoStack`	Latest status update is undone first	O(1) push/pop/peek, O(n) space
Binary Search Tree	`bst.c`	`BSTNode`, `ComplaintBST`	Indexes every complaint by ID; in-order gives sorted output	O(log n) avg, O(n) worst
Bubble Sort	`search_sort.c`	`bubble_sort()`	Adjacent swaps with early termination	$O(n^2)$ avg/worst, O(n) best, O(1) space
Selection Sort	`search_sort.c`	`selection_sort()`	Selects successive minimum values in place	$O(n^2)$ comparisons, O(n) swaps
Insertion Sort	`search_sort.c`	`insertion_sort()`	Inserts each item into a sorted prefix	$O(n^2)$ worst, O(n) best
Merge Sort	`search_sort.c`	`merge_sort()`	Stable divide-and-conquer sorting	O(n log n), O(n) space
Quick Sort	`search_sort.c`	`quick_sort()`	In-place Hoare partitioning, smaller side first	O(n log n) avg, $O(n^2)$ worst, O(log n) space

Detailed Breakdown of Topics, Usage and Logic

1. Linear Search

Where used

- `search_sort.c` (`linear_search`, with match functions `match_name`, `match_area`, `match_id`).

- Called from the Search menu in `main.c` to find complaints by citizen name or area, and used by the benchmark as the baseline for ID lookup.

Logic

- Starts at index 0 and steps through every complaint up to n − 1.
- Calls a match function through a function pointer (for example, "does the name contain this text, ignoring case?"). Every match is copied into an output array.
- Needs no ordering or index, which is why it is used for fields that are not the BST key.
- Worst and average time O(n); space O(1).

2. Binary Search

Where used

- `search_sort.c` (`binary_search`).
- Called from the Search menu in `main.c` on the array returned by `bst_inorder()`, which is already sorted by complaint ID.

Logic

- Operates only on an array sorted by complaint ID.
- Keeps `low = 0` and `high = n - 1`. At each step it computes `mid = low + (high - low) / 2` (written this way to avoid integer overflow) and compares the middle ID with the target.
- If equal, the complaint is returned. If the middle ID is smaller, `low = mid + 1`; if larger, `high = mid - 1`. An empty range returns `NULL`.
- Halves the range every iteration: O(log n) time and O(1) extra space for the iterative version.

3. Singly Linked List

Where used

- `status_list.c` (`StatusNode`, `StatusHistory`).
- One `StatusHistory` is embedded inside every `Complaint` in `complaint.h`.

Logic

- Each node holds `status`, `remarks`, `day` and a single `next` pointer.
- `history_append()` links a new node after `tail` and moves `tail`, so adding a status takes O(1) time with no array resizing.
- `history_display()` walks from `head` through `next` until `NULL`, printing the exact order in which things happened.

- `history_remove_last()` (used by undo) walks to the node before `tail`, frees the tail and moves `tail` back: O(k) with $k \leq 4$. The first node, "Filed", can never be removed.

4. Queue Using Linked Lists (FIFO)

Where used

- `queues.c` (`QueueNode`, `LinkedQueue`).

Logic

- Keeps `front` and `rear` pointers and a `size` counter.
- `queue_enqueue()` links a node after `rear` in O(1). `queue_dequeue()` unlinks and frees the node at `front` in O(1), and resets `rear` to `NULL` when the queue becomes empty.
- Guarantees fairness: among complaints of the same priority, the one filed earlier is always processed first.

5. Priority Handling with Three Queues

Where used

- `queues.c` (`PriorityQueueManager`).
- Called through `grievance_system.c` when a complaint is filed (enqueue) and from `main.c` when "Process next complaint" is chosen (`pqm_dequeue_next`).

Logic

- Holds one `LinkedQueue` each for Urgent, High and Normal in an array indexed by the `Priority` enum.
- `pqm_enqueue()` places the complaint in the queue matching its priority. `pqm_dequeue_next()` checks Urgent, then High, then Normal, and returns the front of the first non-empty queue.
- Urgent complaints are never stuck behind routine ones, and order inside each level stays strictly first-in, first-out. No heap is needed because the number of priority levels is fixed.

6. Stack Using Linked Lists (LIFO)

Where used

- `undo_stack.c` (`StackNode`, `UndoStack`).
- Integrated in `main.c`: `update_status` pushes, `undo_last` pops.

Logic

- `top` points to the most recent node. `undo_push()` allocates a node whose `next` is the old top and makes it the new top, in O(1).

- Every status update pushes a record (complaint ID, previous status, new status).
- Undo pops the most recent record, finds the complaint through the BST and calls `history_remove_last()` on its status list. Last in, first out means the latest change is always reversed first.
- `undo_peek()` and `undo_is_empty()` are also O(1).

7. Binary Search Tree

Where used

- `bst.c` (`BSTNode`, `ComplaintBST`).
- One tree holds every complaint; `main.c` uses it for ID search, status updates, undo and history.

Logic

- Each node points to one complaint. Smaller IDs go to the left subtree, larger IDs to the right.
- `bst_insert()` walks down iteratively and rejects duplicate IDs. `bst_search()` follows a single root-to-node path.
- `bst_inorder()` visits left, node, right, which always yields complaints sorted by ID with no separate sorting step. This array also feeds binary search.
- Randomly generated complaint IDs help reduce highly unbalanced insertion patterns, but the BST does not guarantee balance. Inserting IDs in sorted order would turn it into a straight line and degrade operations to O(n); a unit test covers this behaviour.
- Average search and insert: O(log n); worst case O(n).

8–12. Five Sorting Algorithms

Implemented in `search_sort.c` and selectable in the Sort and view reports menu. Every sort has the same signature, `void sort(Complaint **items, int n, CmpFn cmp)`, sorts the array in place and orders by a comparator function pointer. The menu always sorts a copy, so the original data is never modified.

- **Bubble Sort (`bubble_sort`):** Repeatedly compares adjacent elements and swaps them if they are out of order. Stops early if a full pass makes no swap (O(n) best case). Worst and average: $O(n^2)$.
- **Selection Sort (`selection_sort`):** Splits the array into a sorted prefix and an unsorted suffix. Each pass finds the smallest element in the unsorted part and swaps it into place. Only O(n) swaps, but $O(n^2)$ comparisons.
- **Insertion Sort (`insertion_sort`):** Builds the sorted array from left to right, shifting larger elements right until the correct slot opens. Adaptive: O(n) on nearly sorted data, $O(n^2)$ worst case.

- **Merge Sort (`merge_sort`):** Splits the range at `mid = lo + (hi - lo) / 2`, sorts each half recursively and merges them into a temporary buffer. Takes the left element when keys are equal, so equal keys keep their order (stable). Guaranteed O(n log n) with O(n) extra space.
- **Quick Sort (`quick_sort`):** Divide-and-conquer with Hoare partitioning around the middle element, in place. Recurses into the smaller partition first and loops over the larger one, keeping recursion depth at O(log n). Average O(n log n), worst case $O(n^2)$. Hoare partitioning handles many equal keys, such as equal pending days, without degrading.

Reports can be sorted by pending days (longest first), area, complaint ID or priority. The chosen algorithm and its running time in milliseconds are printed under every report.

IMPLEMENTATION DETAILS

1. Features Implemented

- Console application in standard C99 with no external libraries; every data structure is built by hand with `malloc` and pointers.
- Complaint registration with a generated unique ID, citizen name, area, department and priority (Urgent / High / Normal).
- Three FIFO queues, one per priority, with a "process next complaint" action that always serves the highest non-empty priority.
- A singly linked list per complaint recording the status trail: Filed → Assigned → In Progress → Resolved, with dates and remarks.
- Undo of the last status update using a linked-list stack.
- A binary search tree keyed by complaint ID for lookup and sorted listing.
- Search by ID (BST and binary search) and by citizen name or area (case-insensitive linear search).
- Reports sorted by pending days, area, complaint ID or priority with five algorithms, all using the same comparator-based interface.
- A benchmark that times all five sorts and all three search methods on the same random data.
- 12 pre-loaded sample complaints so every feature can be demonstrated immediately.
- Save and load of all complaints with their full status history to a plain text file.
- 29 automated unit tests, and complete memory clean-up on exit (checked with AddressSanitizer).

2. Menu Options

Option	Purpose
`1` File a new complaint	Register a complaint, generate its ID, add it to the BST and the right queue
`2` Process next complaint	Dequeue the highest-priority complaint and mark it "Assigned"
`3` Update complaint status	Move a processed complaint to In Progress or Resolved, with optional remarks
`4` Undo last status update	Pop the stack and remove the latest status
`5` Search complaints	Search by ID (BST), ID (binary search), citizen name or area (linear search)
`6` View status history	Print the full status trail of one complaint

Option	Purpose
`7` Sort and view reports	Sort by pending days, area, ID or priority with any of the five algorithms
`8` Benchmark	Time all sorts and searches on random data
`9` Save complaints to a file	Write every complaint and its status history to a text file (default `civicdesk_data.txt`)
`10` Load complaints from a file	Replace current data with a saved file; invalid files are rejected and nothing changes
`0` Exit	Quit the program

3. Main Files

File	Purpose
`main.c`	Menu-driven interface (input handling, printing, menu actions)
`grievance_system.h/.c`	Connects BST, queues, undo stack and complaint array; owns and frees them; saves and loads data files
`complaint.h/.c`	Complaint record (owns a StatusHistory), priorities, departments
`status_list.h/.c`	Singly linked list for each complaint's status history
`queues.h/.c`	LinkedQueue (FIFO) and PriorityQueueManager
`undo_stack.h/.c`	Linked-list stack used for undo
`bst.h/.c`	Binary Search Tree of complaints keyed by ID
`search_sort.h/.c`	`linear_search`, `binary_search`, five sorts, comparators, benchmarks
`sample_data.h/.c`	12 sample complaints and a random complaint generator for benchmarks
`util.h/.c`	Safe allocation, string helpers, date conversion
`test_civicdesk.c`	29 automated tests
`Makefile`	Builds `civicdesk` and `test_civicdesk`
`index.html`	Web demo (JavaScript version of the same logic)

4. Data File Format

Option 9 writes a plain text file with fields separated by the `|` character:

```
CIVICDESK 1
C|4821|Ravi|Shadnagar|Water Supply|0|20753
S|1|20755|Assigned to Water Supply department
C|1307|Anita|Gachibowli|Electricity|1|20750
```

- `C` lines hold one complaint: ID, name, area, department, priority (0 Urgent, 1 High, 2 Normal) and the filing day.

- `S` lines hold a later status of the complaint above: status (1 Assigned, 2 In Progress, 3 Resolved), the day and the remarks. The first status, Filed, is implied by the `C` line.
- Days are day numbers (days since 1 January 1970). The `|` character and line breaks are replaced with spaces when saving.
- On loading, the whole file is checked first. If any line is invalid, nothing is changed.

5. Building and Running

A C99 compiler (GCC, Clang or MinGW) is the only requirement.

```
git clone https://github.com/pulegarisashikirran/civicdesk-grievance-
system.git
cd civicdesk-grievance-system
make
./civicdesk

# automated tests
make test
```

Without Make, the program can be compiled directly:

```
gcc -std=c99 -Wall -Wextra -O2 -o civicdesk main.c util.c status_list.c
    complaint.c bst.c queues.c undo_stack.c search_sort.c sample_data.c
    grievance_system.c
```

For an optional memory check, compile with `-g -fsanitize=address,undefined` and run the program.

COMPLEXITY SUMMARY

Searching Algorithms

Algorithm	Best	Average	Worst	Space
Linear Search	O(1)	O(n)	O(n)	O(1)
Binary Search (iterative)	O(1)	O(log n)	O(log n)	O(1)
BST Search	O(1)	O(log n)	O(n)	O(1)

Sorting Algorithms

Algorithm	Best	Average	Worst	Auxiliary space
Bubble Sort	O(n)*	$O(n^2)$	$O(n^2)$	O(1)
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)**

* Bubble sort reaches O(n) only with the early-exit check on already sorted data.

Recursion always continues into the smaller partition first, so the stack depth stays O(log n

TESTING AND VERIFICATION

All **29 automated tests pass**. They cover the data structures, searching, sorting, queue priorities, undo operations and file persistence. In particular, the tests check that:

- every sorting algorithm produces the same order as a reference sort on arrays of 0, 1, 2, 50 and 400 items for every sort field;
- quick sort handles sorted and reversed input, and merge sort is stable;
- BST search and binary search agree for every complaint, and duplicate IDs are rejected;
- queues serve Urgent before High before Normal and keep first-in-first-out order inside a level;
- undo restores earlier statuses in reverse order and never removes “Filed”;
- the system grows past its initial capacity, and an empty system, an empty tree, the queue counts and the history dates after an undo behave correctly;
- saving and then loading a data file restores every complaint, its status history and the queue order, and invalid or missing files, including future-dated complaint and status records, are rejected without changing the current data.

A full pass through every menu option was also run, including invalid input and end-of-input handling. A memory check with AddressSanitizer confirmed that every node and complaint is freed on exit.

BENCHMARK RESULTS

CivicDesk includes a benchmarking module that compares the five sorting algorithms and three searching methods on randomly generated complaint data. All times are measured from the C application, in milliseconds, and may vary with hardware, compiler optimisation and system load.

Sorting Performance

Complaints (n)	Bubble	Selection	Insertion	Merge	Quick
1,000	39.00 ms	32.00 ms	17.00 ms	1.00 ms	1.00 ms
5,000	1,077.00 ms	1,030.00 ms	529.00 ms	5.00 ms	6.00 ms
10,000	6,354.00 ms	5,352.00 ms	2,523.00 ms	18.00 ms	16.00 ms

Searching Performance

500 lookups in a dataset of 20,000 complaints.

Searching algorithm	Execution time
Linear Search	133.00 ms
Binary Search	0.00 ms
BST Search	1.00 ms

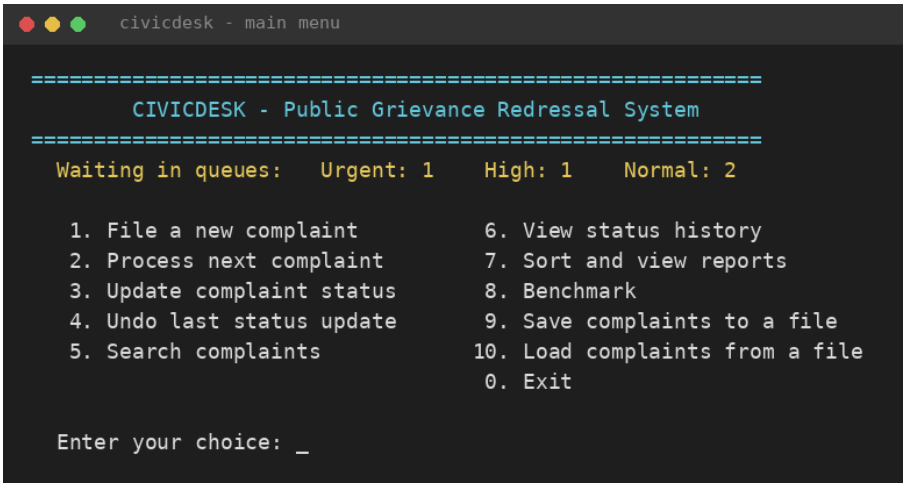
Binary Search showing 0.00 ms means its measured time was below the timer's displayed precision. The BST height in this run was 35.

Performance Analysis

- **Bubble and Selection Sort:** time grows sharply with n because of $O(n^2)$ complexity.
- **Insertion Sort:** performs better than Bubble and Selection Sort in these measurements.
- **Merge and Quick Sort:** stay near O(n log n); at 10,000 complaints they finish in under 20 ms, against seconds for the quadratic sorts.
- **Binary and BST Search:** give efficient lookup (O(log n) on sorted data; O(log n) average and O(n) worst case for the BST), compared with 133 ms for 500 linear searches.

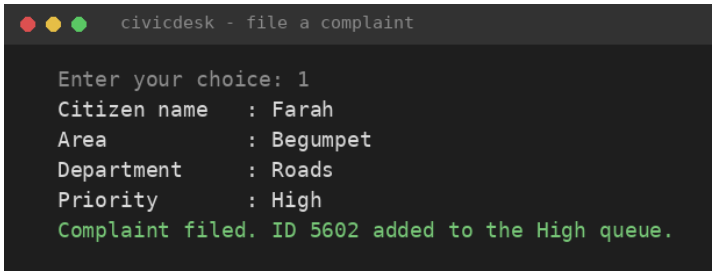
SAMPLE OUTPUTS

1. Main Menu

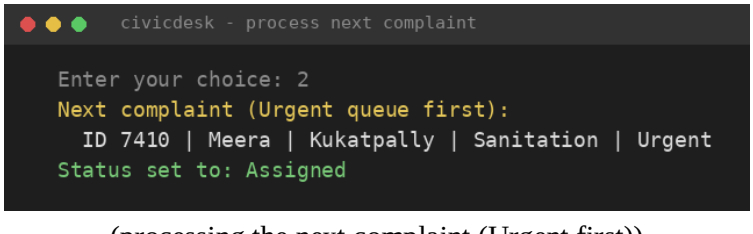


(the main menu, showing how many complaints wait in each queue)

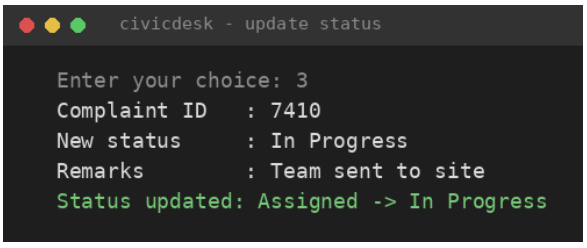
2. Complaint Management



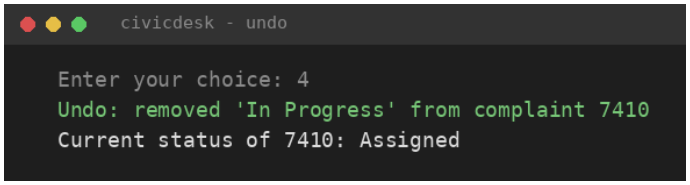
(filing a new complaint)



(processing the next complaint (Urgent first))

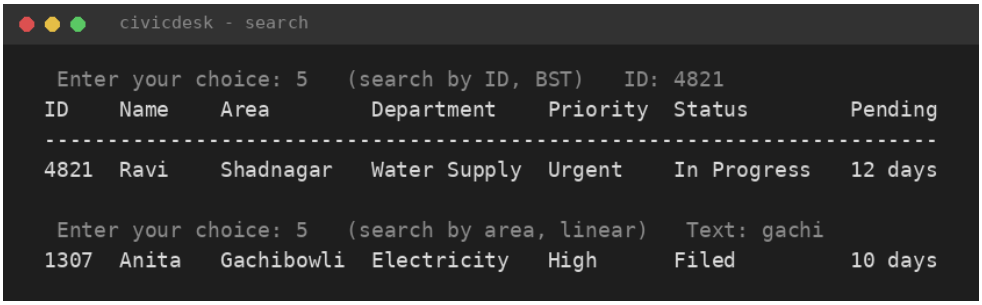


(updating the status of a complaint)



(undoing the last status update)

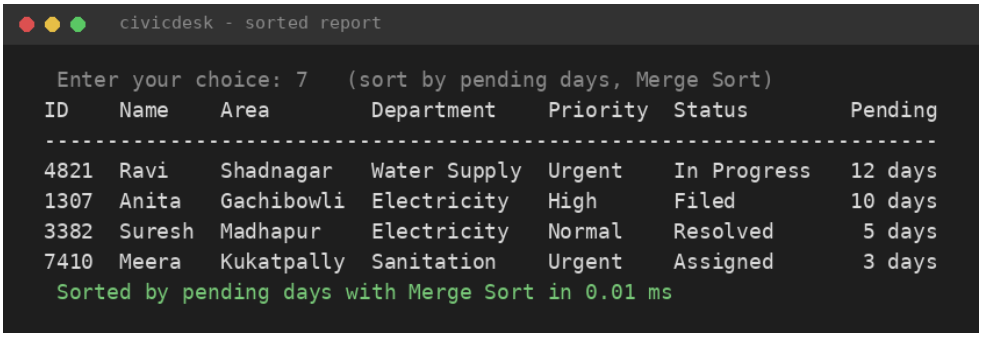
3. Search and Reports



(search by ID and by area)

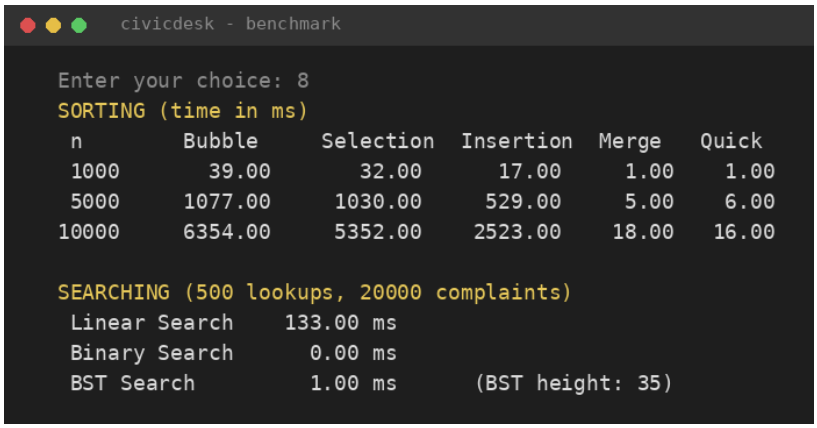


(status history of a complaint)



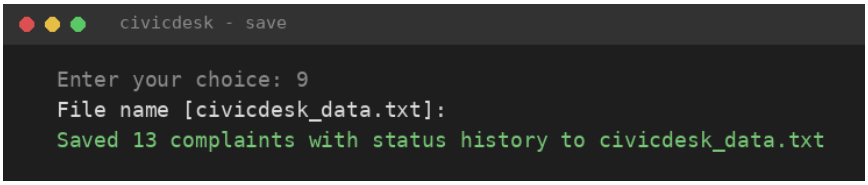
(a report sorted by pending days, with the running time)

4. Performance Benchmark

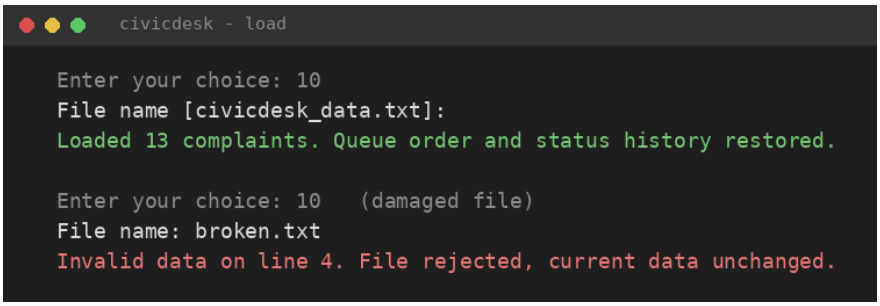


(sorting and searching times from menu option 8)

5. Data Persistence



(saving complaints to a file)



(loading them back, and rejecting a damaged file)

LIMITATIONS AND FUTURE SCOPE

Limitations

- Data lives in memory and is lost when the program closes unless it is saved with option 9. Several users or programs cannot share the data.
- The BST is not self-balancing.
- Priority has only three fixed levels.
- Undo covers status updates only; filing and processing a complaint cannot be undone.
- Text fields have fixed maximum lengths (names and areas up to 31 characters, department up to 23, remarks up to 63). Console input longer than its buffer is rejected rather than silently accepted.
- The interface is a console menu, so citizens cannot file and track complaints themselves.

Future Scope

- A server and database so that data can be shared between programs and users.
- A self-balancing tree (AVL or Red-Black) to guarantee O(log n) even for sorted IDs.
- A heap-based priority queue for finer ranking, for example by pending days within a priority.
- A web or desktop front end with alerts, so citizens can file and track complaints.
- More tests for the menu input handling.

CONCLUSION

CivicDesk is an educational demonstration of queues, singly linked lists, stacks, binary search trees, linear and binary search and five classical sorting algorithms, applied to a realistic public-service problem and implemented in C with manual memory management. Urgent complaints are served first, every status change is recorded and can be undone, and complaints can be found and ranked quickly, as the benchmark shows.

The system is a classroom demonstration and not a replacement for a real government grievance portal. It does not handle real citizen data. The project is released under the MIT License.